Button Smash

Year 5/6 Digital Technologies | Lesson 2 | 45 minutes

Students build a 10 second button smashing game using only the micro:bit's own A and B buttons. The hardware stays familiar so all the thinking goes into the code, and the limitations they hit become the reason to bring Jacdac modules back in next lesson.

Learning Intention and Success Criteria

Learning Intention:

To use variables to keep score and to control when our game is running.

Success Criteria:

I can:

- use a variable to count how many times a button is pressed.
- use a true or false variable so that presses only count while the game is running.
- test my game, find a problem, and fix it.

The constraint that shapes this lesson

> Students are coding on iPads and downloading over **Bluetooth**, which is slow. Every download costs real minutes of a 45 minute lesson.
>
> So the rule for today is **simulator first**. The whole program gets built and tested with the on-screen micro:bit before anyone downloads. Budget **two downloads** per pair: one for the working game, one for an extension if they get there.

Run sheet

Time	Phase	What happens
0 to 5	Hook	Play the finished game yourself on the projector. Take challenges from two students. Do not explain the code yet. Share the learning intention and success criteria.
5 to 12	Design the algorithm	Whole class, no devices. What does the program actually need to do? Build the plain-language steps on the board together. Push them until someone realises the program needs to know whether the game is running, otherwise presses count before it starts. Name that a flag.
12 to 28	Build in the simulator	In pairs. Build the core game and test it entirely with the on-screen micro:bit. Checkpoints on the board: (1) score counts up, (2) score resets on B, (3) presses stop counting after 10 seconds. No downloading until all three work.

Time	Phase	What happens
28 to 36	Download and play	One download each, then actual smashing. Rotate pairs so everyone gets a turn. Record scores on the board as a class leaderboard.
36 to 42	Fairness and debugging	Is the game fair? Students will have found the cheats and the bugs. Discuss, then let them fix one. Fast finishers take an extension and use their second download.
42 to 45	Pack up and hook	Thirty second demo: swap the A button block for a Jacdac button module. Same game, better button. That is next lesson.

The algorithm, in student language

This is what should end up on the board before anyone touches an iPad. Students then translate it into blocks themselves.

- At the start, the score is 0 and the game is not running.
- When B is pressed, set the score back to 0, count down, then set the game to running.
- While the game is running, every press of A adds 1 to the score.
- After 10 seconds, set the game to not running and show the score.

The code

Blocks are what students build. The JavaScript below is for your own reference copy and for projecting if a pair gets stuck. In MakeCode you can paste it into the JavaScript view and switch back to Blocks to see the finished block layout.

Core game

```
let score = 0
let playing = false

input.onButtonPressed(Button.B, function () {
    score = 0
    basic.showNumber(3)
    basic.showNumber(2)
    basic.showNumber(1)
    basic.showString("GO")
    basic.clearScreen()
    playing = true
    basic.pause(10000)
    playing = false
    basic.showString("TIME")
    basic.showNumber(score)
})

input.onButtonPressed(Button.A, function () {
    if (playing) {
        score += 1
    }
})
```

> **Why this works:** the pause inside the B handler does not freeze the whole program. Each event handler runs independently, so presses of A are still counted while B's handler is waiting. Worth telling the class once they ask, because it looks wrong.

Blocks checklist for students

Block drawer	What to find
Variables	Make a variable called score. Make a second one called playing.
Input	on button A pressed. on button B pressed.
Logic	if then. The true and false blocks for setting playing.
Basic	show number, show string, clear screen, pause.

Bugs and cheats they will find

These are the moments the lesson is actually for. Let students discover them rather than pre-empting any of them.

What happens	Why	What to do
Holding A down does not rack up points	A press is one event. The code responds to the moment of pressing, not to the button being down.	Leave it. This is correct behaviour and a good discussion about what an event is.
Pressing B mid-game restarts it	There is nothing stopping the B handler running while a game is already going.	Real bug. Fix by wrapping the B code in if not playing.
Score keeps climbing after TIME shows	The playing variable was never set back to false, or the if block around the A press is missing.	Real bug. This is the success criterion in action.
Everyone gets wildly different scores	Technique, not code. Two thumbs beats one finger.	Use it. Agree on a fair technique as a class, then re-run the leaderboard.

Extensions for fast finishers

1. High score. Add a third variable and show it on the V2 logo touch, not on shake, because shake triggers during smashing.

```
let best = 0

// add this just after playing = false
if (score > best) {
    best = score
    basic.showString("BEST")
}

input.onLogoEvent(TouchButtonEvent.Pressed, function () {
    basic.showNumber(best)
})
```

2. Random game length, so you cannot pace yourself. Replace the pause with a random wait between 5 and 15 seconds.

```
basic.pause(randint(5000, 15000))
```

3. Two player over radio. Both micro:bits need the same group number. Each sends its score at the end and compares.

```
radio.setGroup(1)

// add this just after showing the score
radio.sendNumber(score)

radio.onReceivedNumber(function (receivedNumber) {
    if (score > receivedNumber) {
        basic.showString("WIN")
    } else if (score < receivedNumber) {
        basic.showString("LOSE")
    } else {
        basic.showString("DRAW")
    }
})
```

Looking after the hardware

> Set the rule before the first download: **board flat on the desk, fingertip taps, no fists and no table banging**. The micro:bit's own buttons are small tactile switches and a class of Year 5/6 students smashing them will cost you boards.
>
> This is also the honest argument for next lesson. A Jacdac button module is bigger, sturdier and built to be pressed hard, so the game gets better and the kit survives.

Where this leads

Next lesson is a single block swap: on button A pressed becomes on button1 pressed from the Jacdac button module. Students already know how to add the extension and the module blocks from the intro lesson, so the swap takes minutes and the rest of the time goes into making the game better.

From there the LED ring they lit up in the intro lesson becomes the score display, which gives a reason to map a score onto a number of lights. That mapping is the same idea the sound bender projects rest on.